

Instructions

- Homework 1 was due Monday, July 27th at 18:00 Chicago Time.
 - All numbers and figures come from `homework_1_solution.ipynb`, run from top to bottom.
-

Problem 1: One-Dimensional Data

Load in the data from the GitHub repository for this class.

```
1 import pandas as pd
2
3 df = pd.read_csv(
4     "https://raw.githubusercontent.com/tobiasdelpozo/data-analysis-2026/refs/heads
5     /master/homework/homework_1/questions/homework_1_data.csv"
```

Problem 1.1

For the feature labeled **X1**, compute the mean, median, variance, and standard deviation. Report your numbers below (rounded to at least 4 decimal places).

Answer:

	Value
mean	12.3435
median	11.9216
variance	38.8894
std	6.2361

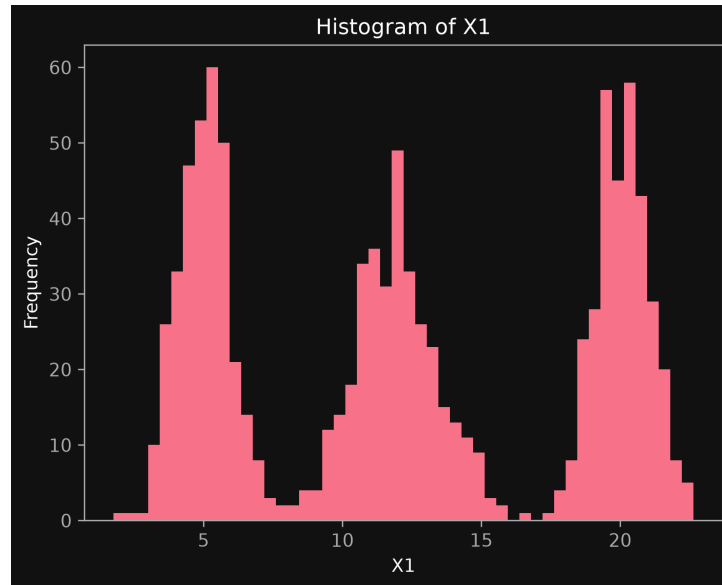
Problem 1.2

Display a histogram of the feature **X1** using 50 bins. Do you think that the statistics you computed in 1.1 are good descriptors of the data? Include the graph below, and explain your reasoning in 1-2 sentences.¹

Answer:

No. The data has three peaks, at roughly 5, 12 and 20, each holding about a third of the points. The mean and the median land on the middle peak and say nothing about the other two, and the standard deviation of 6.2361 measures the gaps between the peaks rather than the spread within any one of them.

¹Hint: you can use `\includegraphics{}` to include images in L^AT_EX.



Problem 1.3

Using the same feature **X1**, come up with some metrics that are descriptive of the distribution of the data. Note, this is open-ended, so think about what the data looks like, and how a human would describe it.

Answer:

Take the local maxima of a kernel density estimate as the modes, then assign each point to its nearest mode and describe the three groups separately.

cluster	mode	mean	std	share
0	5.0392	5.0368	0.9878	0.3320
1	11.8508	11.9600	1.4572	0.3370
2	20.0624	20.0628	0.9787	0.3310

Within a cluster the standard deviation is about 1, against the 6.2361 of 1.1. Four numbers per mode, plus the three shares, describe the data; one mean does not.

Problem 2: kNN Regression

This problem uses the same dataset as Problem 1.

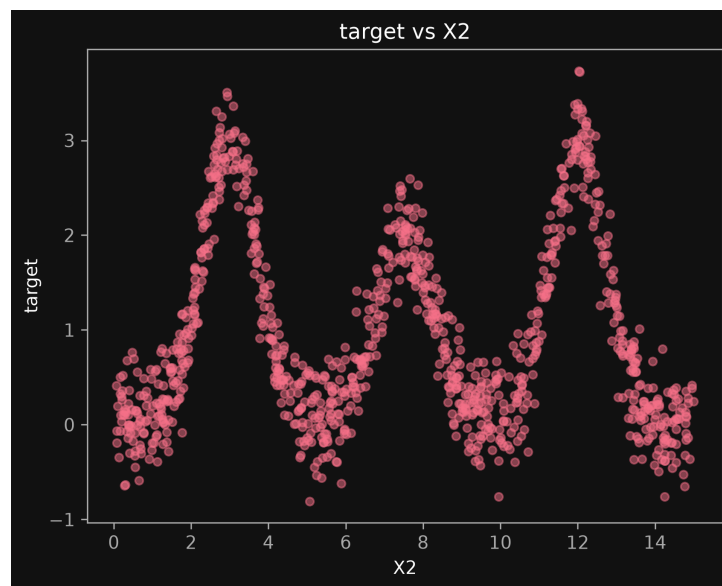
We're going to implement a k-Nearest Neighbors regression model. Unless otherwise specified, use an 80/20 train/test split for all parts of this problem.

Problem 2.1

Display a plot of X_2 versus the **target** variable. What do you notice about the relationship between these two variables?

Answer:

The relationship is not linear. The target rises and falls three times across the range of X_2 , peaking near $X_2 = 3, 7.5$ and 12 , and the noise around that shape is small.



Problem 2.2

Implement a kNN regression model from scratch. You may use `numpy` and `pandas`, but you may not use any machine learning libraries (e.g. `scikit-learn`).

Your model should take in 4 parameters:

- `X_train`: training features
- `y_train`: training target variable
- `X_test`: testing features
- `k`: number of neighbors to use

And it should output the predicted values for `X_test`.

The algorithm you should use is as follows:

1. For each test point, compute the Euclidean distance to all training points.
2. Identify the k-nearest neighbors based on these distances.
3. Compute the predicted value as the mean of the target variable of these k-nearest neighbors.

Note that we are only considering a single feature for this problem, so the Euclidean distance is simply $\sqrt{(x_{\text{test}} - x_{\text{train}})^2}$.

Include your code implementation below.²

Answer:

Two implementations. The first loops over test points; the second does the whole distance matrix at once, which is faster but costs $n_{\text{train}} \times n_{\text{test}}$ memory.

```

1 def knn_regressor(X_train, y_train, X_test, k):
2     y_pred = []
3     for x_test in X_test:
4         distances = np.linalg.norm(X_train - x_test, axis=1)
5         knn_indices = np.argsort(distances)[:k]
6         y_pred.append(np.mean(y_train[knn_indices]))
7     return np.array(y_pred)
8
9
10 def knn_regressor_matrix(X_train, y_train, X_test, k):
11     distances = np.linalg.norm(X_train[:, np.newaxis] - X_test[np.newaxis, :],
12                               axis=2)
13     knn_indices = np.argsort(distances, axis=0)[:k, :]
14     return np.mean(y_train[knn_indices], axis=0)

```

The two agree to floating point on this data.

Problem 2.3

Randomly split the data into training and testing sets (80/20 split), and report the Mean Squared Error (MSE) of your kNN regression model on the test set for $k = 5$.

Answer:

MSE = 0.0996 at $k = 5$.

```

1 np.random.seed(42)
2
3 ind = df.index.to_numpy().copy()
4 np.random.shuffle(ind)
5 n = len(ind)
6 train_ind, test_ind = ind[:int(n * 0.8)], ind[int(n * 0.8):]
7
8 X_train = df.loc[train_ind, ["X2"]].to_numpy()
9 X_test = df.loc[test_ind, ["X2"]].to_numpy()
10 y_train = df.loc[train_ind, "target"].to_numpy()
11 y_test = df.loc[test_ind, "target"].to_numpy()
12
13 y_pred = knn_regressor(X_train, y_train, X_test, k=5)
14 mse = ((y_test - y_pred) ** 2).mean()

```

²Hint: you can use `\lstlisting[language=python]` to include Python code snippets.

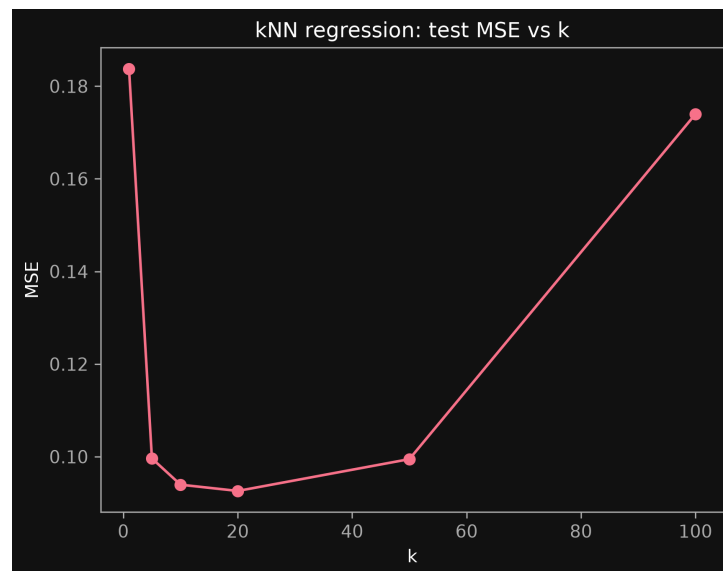
Problem 2.4

For $k \in \{1, 5, 10, 20, 50, 100\}$, compute the MSE on the test set and plot the results (k values on the x-axis, MSE on the y-axis). What value of k gives the best performance on the test set?

Answer:

$k = 20$, at $\text{MSE} = 0.0926$.

k	test MSE
1	0.1838
5	0.0996
10	0.0939
20	0.0926
50	0.0995
100	0.1740

**Problem 2.5**

Which value of k do you think has the highest bias? And which has the highest variance? Explain your reasoning in 1-2 sentences.

Answer:

$k = 100$ has the highest bias. It averages an eighth of the 800 training points for every prediction, which flattens the three peaks of 2.1 into something close to the overall mean.

$k = 1$ has the highest variance. Each prediction is a single training point, so it carries that point's noise in full.

Both sit at the ends of the table above, and both are worse than $k = 20$.

Problem 3: Linear Regression

Using the same dataset as Problems 1 and 2, we are going to explore linear regression.

Problem 3.1

Using `statsmodels`, fit a linear regression model to predict `y` using `X3`. You should **not** use an intercept term in your model. Report your β coefficient below:

Answer:

$$\hat{\beta} = 0.5642.$$

Problem 3.2

Re-run the linear regression model from 3.1, but this time include an intercept term. What are your new β coefficients (intercept and slope)?

Answer:

$$\hat{\beta}_0 = 7.1515 \text{ and } \hat{\beta}_1 = -0.2926, \text{ with } R^2 = 0.1707.$$

The slope changes sign once the intercept is free. Without an intercept the line is forced through the origin, and since y is around 5 everywhere, the only way to fit it is with a positive slope.

Problem 3.3

Do the following data transformations:

$$\tilde{y} = y - \bar{y} \quad \tilde{X}_3 = X_3 - \bar{X}_3$$

Re-run the linear regression model using $\tilde{y}$ and $\tilde{X}_3$, without an intercept term. What is your β coefficient? How does it compare to your answer in 3.1?

Answer:

$$\hat{\beta} = -0.2926. \text{ This does not match 3.1, but it matches the slope in 3.2 exactly.}$$

Centering is the same thing as fitting an intercept. The OLS intercept is

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{X}_3,$$

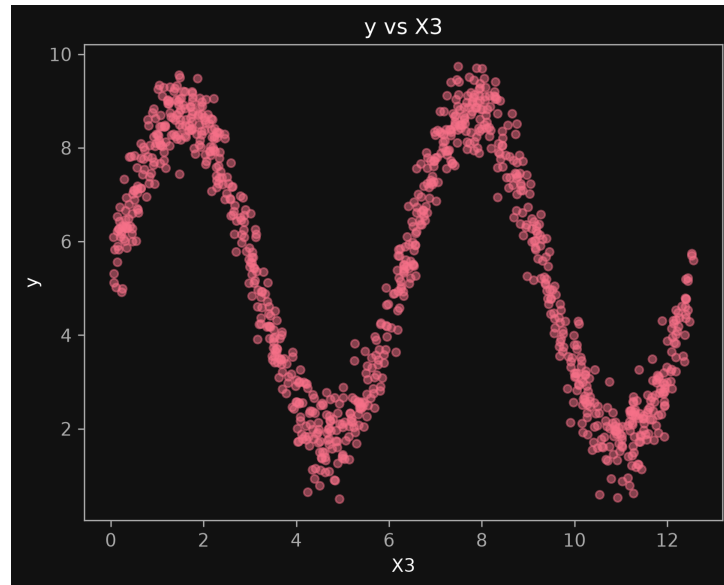
so subtracting the means from both sides leaves a model whose intercept is zero by construction.

Problem 3.4

Inspect your data. Display a scatter plot of `X3` versus `y`. What do you notice about the relationship between these two variables? Is a linear model appropriate for this data? Explain your reasoning in 1-2 sentences.

Answer:

The relationship is periodic: y runs through two full cycles between 5 and 9 as X_3 goes from 0 to about 4π . A straight line cannot follow that, which is why 3.2 explains only 17% of the variance.

**Problem 3.5**

Define a new feature `X3_sin` as follows:

$$\text{X3_sin} = \sin(\text{X3})$$

Fit a linear regression model to predict `y` using `X3_sin`, including an intercept term. Report your β coefficients (intercept and slope) below:

Answer:

$\hat{\beta}_0 = 5.2484$ and $\hat{\beta}_1 = 3.5362$, with $R^2 = 0.9639$ against the 0.1707 of 3.2.

Problem 3.6

Display a plot of `X3_sin` versus `y`. Do you think a linear model is appropriate for this data? Explain your reasoning in 1-2 sentences.

Answer:

Yes. The transformation makes the relationship linear, which is the point: the model is still linear in its coefficients, and all we changed was the feature.

